

CLOUDFLARE · ASSOCIATE SOLUTIONS ENGINEER

Take-Home Assessment

整体设计与开通流程

一套完全运行在免费套餐上的 Cloudflare 全栈演示:应用服务 · 零信任 · 开发者平台

候选人:<你的名字> 仓库:github.com/LiCarry/cloudflare 2026-09

方案总览:一个应用,三层能力

以 Render 上的销售分析 Dashboard 为业务载体,把评估要求的三大板块串成一条完整链路

1 应用服务 Application Services

- 源站部署于 Railway 免费额度(Express + 静态 Dashboard)
- 域名接入 Cloudflare,橙云代理全量接管流量
- TLS Full (strict) 端到端加密
- WAF 托管规则集 拦截 SQL 注入(边缘 403)
- 限流规则 保护登录端点(10 次/10 秒)
- 防绕过:直连源站 IP 一律 403

2 零信任 Zero Trust

- Cloudflare Tunnel 以子进程运行于源站,纯出站连接、零入站端口
- SSO 身份源:一次性 PIN(邮件验证码),可扩展 Google / GitHub
- Access 策略 锁定 `/secure` :仅本人 + `@cloudflare.com`

3 开发者平台 Developer Platform

- Worker 渲染身份页:验证 Access JWT 后输出邮箱 / 时间 / 国家
- 国旗资产:私有 R2 桶(零公开 URL)与 D1 数据库 双通道提供
- 全程 Wrangler CLI 创建、部署、绑定

整体架构:三条流量路径,一个边缘

所有流量先经过 Cloudflare 边缘(WAF · 限流 · Access),再按路径分流到三种后端



安全设计:四道防线层层递进

从加密、攻击面收敛、滥用防护到绕过兜底,每层都有可演示的证据

加密 TLS:为什么推荐 Full (strict)

- **Flexible**:浏览器有锁,但 CF→源站是明文 HTTP —— 最后一步裸奔
- **Full**:加密但不校证书,自签名证书的中间人仍可行
- **Full (strict)** :加密且校验;Render 提供 `*.onrender.com` 有效证书,零配置达标

WAF 托管规则 + SQL 注入拦截

- 开启 **Cloudflare 托管规则集**(免费版内置),另加自定义规则匹配 `union select / or 1=1 / sleep(`
- 演示:攻击请求边缘返回 **403**,响应头 `cf-mitigated-header: block`,源站零感知

滥用 限流:保护登录端点

- 规则: `/api/login` 同 IP > 10 次 / 10 秒 → 封禁 1 分钟
- 缓解风险:撞库与暴力破解;攻击流量不再消耗源站容量
- 演示:页面"Fire 20 requests"按钮,前 10 次 401 JSON,之后 429

兜底 防绕过:IP 检查 + 共享密钥

- 中间件解析 `X-Forwarded-For` 最右侧不可伪造的对端 IP,仅放行 CF IP 段/环回(Tunnel)
- 发现 Railway 把 XFF 规范化为访客 IP → 加 **Transform Rule** 强制注入 `X-Origin-Secret` 密钥头,源站比对 `CF_SHARED_SECRET`,直连者无从得知
- 生产更强:防火墙仅放行 CF 段 / **Authenticated Origin Pulls(mTLS)**

零信任设计:Tunnel + Access,不给攻击面留门

身份是新的边界 —— 没有正确身份的请求,连源站的门都摸不到

A Cloudflare Tunnel(反向连通)

- Zero Trust 控制台创建**远程管理隧道**,Token 注入 Render 环境变量 `TUNNEL_TOKEN`
- 源站启动时自动拉起 `cloudflared` 子进程,纯出站 QUIC/HTTP2 连接,无需任何入站端口
- 公共主机名 `tunnel.域名` → `http://localhost:PORT` (容器内)
- 子进程带守护重启;二进制在 `postinstall` 自动下载

B 身份与策略

- 登录方式:一次性 PIN(零外部依赖;Google/GitHub OAuth 同样一键接入)
- Access 应用覆盖 `/secure` :Include 规则 = 本人邮箱 OR 邮箱域 = `cloudflare.com`
- 未授权访客在边缘即被拦截登录页,永远到不了源站

C 身份如何传递

```
# Access 放行后注入源站的头
cf-access-authenticated-user-email: you@example.com
cf-access-jwt-assertion: eyJhbGciOiJIJFuzI1NiIs...
cf-ipcountry: CN
```

Worker 设计:身份验证 + 双存储通道

不信任边缘头,自己验签;同一份国旗资产,R2 与 D1 各成一条供给链路

1 /secure:身份页(HTML)

- 取 `Cf-Access-Jwt-Assertion`,用 `WebCrypto` 验签(RS256/ES256 双支持):拉取团队 JWKS(`/cdn-cgi/access/certs`),校验 `kid` / `iss` / `aud` / `exp`
- 国家来自边缘注入的 `request.cf.country`
- 输出(国家为链接): `you@x.com authenticated at 2026-09-07T... from CN`

2 国旗资产:私有 R2

- `/flags/:CC` → R2 binding 读取 `flags/cc.svg`, `Content-Type: image/svg+xml`
- 桶不开启任何公开访问(无 `r2.dev` 域名),资产仅能经 Worker 取出

3 国旗资产:D1 通道

- `/flags-d1/:CC` → D1 binding 参数化查询 `flags` 表(TEXT 存 SVG)
- 响应头 `x-flag-source` 标明供给方,便于演示对比

工程上踩过的坑(及解法)

- D1 单语句上限 100KB,塞尔维亚国旗一个 SVG 就 181KB
→ 空 INSERT + `UPDATE ... content || '分片'` 拼装,回读字节级一致
- 批量导入偶发失败 → `load-d1.sh` 分片重试 + 行数校验
- R2 上传 257 个对象 → 脚本并发 8 路 `wrangler r2 object put`
- Access JWT 实为 RS256(非 ES256),且 Workers WebCrypto 需显式声明 hash → 双算法支持 + `{hash:"SHA-256"}`

Wrangler workflow

```
wrangler r2 bucket create ase-flags
wrangler d1 create ase-flags-db
wrangler deploy # bindings: FLAGS / FLAGS_DB
```

开通流程 i :应用服务(Render → Cloudflare)

约 40 分钟;全部免费套餐,每步都有验证命令

STEP 1

部署源站

Railway 从 GitHub 部署
Networking → Generate
Domain
验证: `curl`
`/healthz`

STEP 2

域名接入

Cloudflare 添加域名
(Free)+ NS 委派
Railway 添加 Custom
Domain
CF 配专用 CNAME +
`_railway-verify`
TXT
橙云代理,验证 `cf-ray`

STEP 3

TLS 加密模式

SSL/TLS → Overview
设为
Full (strict)

验证: `curl -I`
`http://...` 强制跳
HTTPS

STEP 4

WAF 规则

安全 → WAF → 托管规
则:确认
已启用

自定义规则:查询串含
`union select / or 1=1` →
Block

STEP 5

限流规则

安全 → WAF → 限流
`/api/login` · 同 IP
>10 次/10 秒 → Block 1
分钟

STEP 6

防绕过

Transform Rule 注入密钥头
+ Railway 变量
`REQUIRE_CLOUDFLARE=true`
`CF_SHARED_SECRET`
验证:直连 → 403

开通流程 ii:零信任(Tunnel + Access)

约 30 分钟;一次配置,之后每次部署自动重连

STEP 1

创建隧道

Zero Trust → 网络 → Tunnels
创建"远程管理"隧道,复制 Token
粘贴到 Railway 变量 `TUNNEL_TOKEN` 并
重新部署
日志出现 `[cloudflared]` 即连上



STEP 2

公共主机名

隧道配置 → Public Hostname
`tunnel.域名` → HTTP `localhost:`
`<PORT>`
(端口以 Railway 日志为准,本例 8080)
DNS 记录自动创建



STEP 3

身份源

设置 → 身份验证 → 登录方式
添加
一次性 PIN
(免配置)
可选:Google / GitHub OAuth



STEP 4

Access 应用

访问 → 应用程序 → 自托管
域: `tunnel.域名/secure` (再建一个 `域名/secure` 给 Worker)
Include:本人邮箱 OR 邮箱域
`cloudflare.com`

开通流程 iii:开发者平台(Worker + R2 + D1)

约 30 分钟;一条命令一条资源,配置全部收敛在 wrangler.toml

STEP 1

登录 Wrangler

```
npx wrangler
login
cd worker && npm
install
```

STEP 2

R2 私有桶

```
wrangler r2
bucket create ase-
flags
桶名写入 wrangler.toml
保持
无公开访问
```

STEP 3

上传国旗

```
scripts/download-
flags.sh
(257 面 SVG,flag-icons)
node
scripts/upload-
r2.js
```

STEP 4

D1 建库灌数

```
wrangler d1
create ase-flags-
db
database_id 回填 → 建表
(schema.sql)
make-d1-seed.js +
load-d1.sh
```

STEP 5

配置并部署

填
ACCESS_TEAM_DOMAIN
/ ACCESS_AUD
(Access 应用页复制 AUD
标签)
wrangler deploy

STEP 6

路由绑定

Workers 路由: 域
名/secure*
/flags/* ·
/flags-d1/*
→ ase-assessment-
worker

现场演示:五个一气呵成的故事

每一步都是"攻击发生 → Cloudflare 拦截 → 证据可见"

环节	操作	预期证据
① WAF 拦 SQLi	<code>curl -i "https://域名/search?q=' OR 1=1 --"</code> ,再换 <code>?q=coffee</code>	攻击 403 + <code>cf-mitigated-header: block</code> ; 正常查询 200
② 限流生效	打开 <code>/login</code> 点 "Fire 20 requests"(或 curl 循环)	先 401(JSON 来自源站),超阈值后 429(来自 Cloudflare)
③ 防绕过	<code>curl -i https://应用.onrender.com/</code> 对比 <code>curl -i https://域名/</code>	直连 403 解释页;走 Cloudflare 200
④ 零信任登录	隐身窗口访问 <code>tunnel.域名/secure</code> → 邮箱收 PIN 登录	未登录被 Access 拦截;登录后页面显示身份头
⑤ Worker 身份 + 国旗	访问 <code>域名/secure</code> → 点击国家代码 CN → 再看 <code>/flags-d1/CN</code>	"email authenticated at ... from CN ";国旗 SVG 渲染, <code>x-flag-source</code> 区分 R2 / D1

交付物与仓库导航

github.com/LiCarry/cloudflare

- `server.js + lib/` 源站与防绕过中间件
- `public/dashboard.html` 业务 Demo 页
- `worker/` Worker + R2/D1 绑定
- `scripts/` 国旗下载 / R2 上传 / D1 灌数
- `docs/SETUP.md` 逐步开通指南 + 排错表
- `docs/REPORT.md` 书面报告

✓ 免费套餐全覆盖

- Railway 免费额度(源站)· 免费子域 `clouddemo.cc.cd`(NS 委派至 Cloudflare)
- Cloudflare Free:代理 / 托管 WAF / 1 条限流规则
- Zero Trust $\leq$ 50 用户;Tunnel 无限连接
- Workers 免费额度;R2 10GB + D1 5GB

一句话总结

"一个域名、一个免费实例、一条隧道、一个 Worker——把 WAF、限流、零信任和无服务器存储串成了完整的安全链路。"